

Godogen 小白指南

Autonomous Game Development with Claude Code & Codex

描述一句游戏创意 → AI 自动架构、画美术、写代码、跑引擎、看画面质检 → 交付能玩的游戏

~7,000+

GitHub Stars

3

引擎: Godot 4 / Bevy / Babylon.js

2

宿主: Claude Code / Codex

MIT

开源协议

0 行

仓库里的游戏代码

0 • 一分钟速览：这到底是个啥

先纠正直觉：**Godogen 仓库里没有一行游戏代码，也没有游戏引擎**。它是一套精心编写的 **Markdown 指南 + 技能包 (Skill) + 一个发布脚本**。它把这几页"说明书"发布 (publish) 到一个空的游戏仓库里，然后让 Claude Code / Codex **住进**这个仓库，照着说明书把游戏真的做出来。



发布出来的仓库薄得惊人，只有三样东西：**运行时清单** (CLAUDE.md，约 12 行的干活章程)、**引擎指南** (godot.md / bevy.md / babylon.md 单页)、**asset-gen 技能包** (AI 画美术的手艺)。项目脚手架、截图工具——**全部由 AI 到场后按指南现场重建**。

疑问清单：工科毕业生的 12 连问

Q1 仓库里没有游戏代码，凭什么能"产出游戏"？

因为它的产品不是游戏，而是 **"能指挥 AI 做游戏的说明书"**。类比：驾校不发车，但发的教材能让学员自己开走一辆车。Godogen 把"做一个游戏"拆解成 AI 可执行的章程——去哪记进度（README.md）、怎么画美术（asset-gen 技能）、每个引擎怎么搭项目（引擎指南）、做完怎么自证（跑起来看画面）。AI 智能体读到这几页纸，就能自主推进几小时的开发。

这代表一种新物种：源码即 prompt。 仓库的价值全在几篇 Markdown 的措辞里——哪些坑要提前告诉模型、哪些决策留给模型现场判断，都是设计出来的。

Q2 为什么不直接对 Claude Code 说"帮我做个游戏"？

直接说也能做，但质量不稳定。裸prompt的模型会犯一批 **"编译能过、一跑就废"** 的错（后面 Q6 详述），还会在长任务里忘了自己做到哪。Godogen 相当于给 AI 配了三件套：

装备	解决什么
运行时清单 (CLAUDE.md)	固定两件事：持久状态写在 README.md（会话被压缩也不丢进度）；结果必须"跑起来看到"才算数
引擎指南（单页）	只写模型推不出来或查起来慢的：项目骨架、无头截图配方、静默失败陷阱清单
asset-gen 技能包	画图/3D/帧动画的完整工艺 + 计费意识，按需加载不占上下文

官方文档说得直白：**没有固定的多阶段流水线，也没有规定的文档协议**——一个足够强的模型自己会规划分解，说明只需钉死"状态存哪"和"结果怎么证明"这两件事。

Q3 运行时清单只有 12 行，这几段话凭什么指挥 AI 干几小时？

看原文（我发布出来的 CLAUDE.md 实拍）：

```
# Build Godot game from a description
- Keep durable project status in `README.md`
- Generate visual assets with `/asset-gen`
- Read `godot.md` for engine guidance
## Delivery
Judge progress from the running game, never from a clean build
```

← 进度落盘，抗上下文压缩

← 画美术用技能包；花钱前先确认

← 引擎细节按需查单页指南

← 铁律：证据优先

它的威力来自 **"少即是多"的设计哲学**：清单只钉死两条不可妥协的原则（状态存 README、结果看运行），把架构设计、任务分解、技术选型全留给模型现场发挥。模型越强，这套说明越有效——它是给"聪明员工"的手写便条，不是给流水线工人的 SOP。

Q4 "Proof over claims" (证据优先于声称) 是什么意思?

这是整个项目最核心的思想。编译通过 ≠ 游戏能玩。场景里模型可能静默丢节点、材质可能没贴上、物理可能根本没动——这些都反映在编译结果是 0 error。所以清单里写死：进度只能从"正在运行的游戏"判断。



Q5 质检时 AI 怎么"看"画面？我要不要全程盯着？

两条路线，AI 根据你布置任务的说法自己选：

模式	怎么运作	适合
盯直播 (live steering)	Babylon.js 直接给一个浏览器 URL；Godot/Bevy 项目你本机随时能跑。AI 在口味/范围/花钱的决策点暂停问你，其余放开干	开放式探索："做个海战游戏，咱们边看边调"
挂机托管 (unattended)	丢下一份明确的需求书，AI 自己做主稳步推进，跑完交 15~20 秒实机录像；官方建议挂云服务器 + tmux，接 Telegram/Slack 随时查岗	明确需求："完全照这份 spec 做，做完叫我"

截图/录像怎么来：Godot 用 `--write-movie` 电影写入器（`--fixed-fps 30` 让运动完全确定性，450 帧=15 秒），Bevy 离屏渲染，Babylon 用 Chrome WebGL2 截浏览器页面——都是无头（headless）自动采集，不需要人操作。

Q6 引擎指南为什么专挑"静默失败"的坑写？

指南作者的原则：**模型已经会的绝不写，只写"编译能过但运行必坏"的坑**。这类错误没有报错信息，AI 自己撞了墙都不知道墙在哪，是纯 prompt 产品里最值钱的内容。Godot 指南实拍几条：

静默陷阱	后果
节点没设 <code>Owner</code> 链就序列化	节点直接从 <code>.tscn</code> 里消失，不报错
<code>Owner</code> 递归误入 GLB 实例内部	网格全部内联成文本 → <code>.tscn</code> 暴涨 100MB+
程序化网格忘调 <code>GenerateNormals()</code>	影子整体消失，无任何警告
对导入网格用 <code>CreateTrimeshShape()</code>	物理帧率掉到 <1 FPS
目录里误放 <code>.gdignore</code>	导入器静默跳过整个目录（如 <code>assets/</code> ）
C# 枚举名照 GDScript 记忆猜	训练数据偏 GDScript，C# 枚举常猜错

指南还给出对策模式：打包场景前先数节点数，`pack` 后 `Instantiate` 出来再数一遍，数量对不上就拒绝保存——把静默失败改造成会显式报错的门禁。

Q7 场景为什么要"构建期用 C# 脚本生成"，而不是在编辑器里搭？

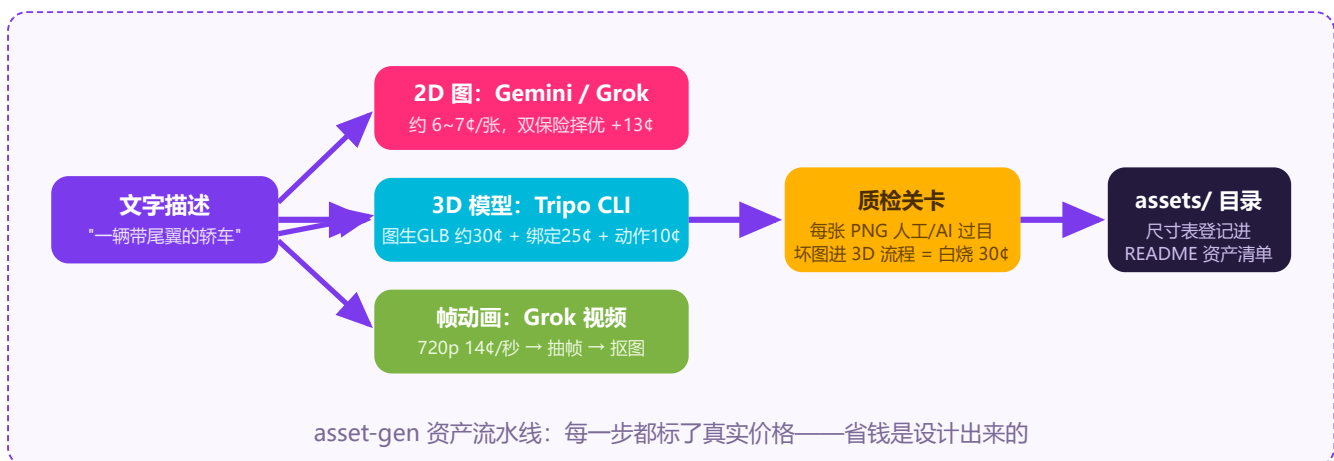
因为 **玩家是 AI，不是人**。人类用编辑器拖拖拽拽；AI 的原生语言是代码。所以 Godot 指南规定：场景写成 **C# SceneTree 构建脚本**，无头跑一次就吐出 `.tscn` 文件——

```
godot --headless --script scenes/BuildX.cs # 构建脚本：搭节点树→设属性→挂脚本→打包保存
```

三条硬规矩：① 构建脚本只管搭积木，不含任何运行时逻辑（没有 `_Ready/``_Process/`信号）；② 先建叶子场景再建父场景；③ 挂脚本必须放最后（`SetScript()` 会销毁 C# 包装对象，顺序错了引用全失效）。这就是"AI 原生开发工作流"的一个具体样本：把人类图形界面工具，翻译成模型擅长的代码生成。

Q8 美术是 AI 画的？整条资产流水线长什么样？

asset-gen 技能包（一个标准的 Claude Code Skill）管全部美术，路线按素材类型分流：



三个省钱妙招：① **贴图集 (kit) 战术**：一次生成费最低 1K 尺寸，那就把 4 个道具画进同一张 1024² 图，再用 `grid_slice.py` 切开（我在实验里实测跑通了）；② 抠图**永远不要在 prompt 里要“透明背景”**（模型会画上棋盘格假透明），要纯色底再抠；③ 本机有显卡就优先走免费的本地 `qwen-image`，付费 API 只留给主角/参考图这类质量关键资产。

Q9 一套源码怎么变出 3 引擎 × 2 宿主 = 6 种口味？

答案是 **"发布期渲染" (publish-time render)**：Markdown 里挖好 `{VAR}` 占位符，发布脚本按你选的引擎+宿主填变量。 `./publish.sh --engine godot --agent claude --out ~/my-game` 一条命令，产出：

变量	claude 口味	codex 口味
清单文件名	CLAUDE.md	AGENTS.md
技能目录	.claude/skills/	.agents/skills/
技能调用写法	/asset-gen	\$asset-gen
额外元数据	不需要	脚本生成 agents/openai.yaml

引擎差异也类似：Babylon 的资产目录渲染成 `src/assets`，Godot/Bevy 是 `assets`；`.gitignore` 内容按引擎分别给。**没有 6 份源码树，只有 1 份模板 + 1 个渲染器**——这是配置即代码（template rendering）的经典手法，我在实验 1/2 里完整复现了它。

Q10 AI 花我的钱画图，成本怎么管得住？

成本控制被写进了章程，三道闸：

- ① **花钱前置确认**：manifest 写死"第一次付费生成前必须向用户确认开销"；② **价格全部明码**：技能文档里每个模型都标单价（1K 图 6~7¢、精灵视频 720p 14¢/秒、Tripo 模型约 30 credits），还要求跑批前先 **tripo balance** 查余额、汇报真实扣费数而非估算；③ **资产登记表带尺寸列**：README 里每个资产必须登记"游戏内尺寸"（"车长 4m"、"精灵 128×128px"）——文档原话：不写这列，写代码的 AI **总是**把素材比例搞错。免费档（本地 qwen-image）优先，付费档留给关键资产，双 key 齐全时关键图两张都生成、择优保留。

Q11 它跟我学过的 Agent Skills / Claude Code 是什么关系？

Godogen 就是知识库已有概念的 **集大成实战案例**：

已学概念	在 Godogen 里的化身
[[AgentSkills技能包]]	asset-gen 就是一个带 SKILL.md frontmatter 的标准技能，按需加载不占主上下文
[[Agent循环]]	"写码→跑引擎→看截图→修复"就是感知-行动循环，把"截图"作为感知输入
[[LLM工具调用]]	AI 调 grid_slice/rembg/tripo CLI 等工具完成美术工艺
[[提示词模板]]	publish 期 \${VAR} 渲染，一套模板出 6 种口味
[[Checkpoint存档与持久执行]]	"持久状态写 README.md" = 用人-readable 文件做会话间的存档点

新增量词：跨智能体通用技能目录（.agents/skills 已是 Codex 等工具的事实标准）、**proof over claims**、**build-time scene generation**。

Q12 我能拿它干什么？本机没装 Godot 怎么办？

实际门槛盘点：**真跑一局完整的 AI 生成游戏**需要 Godot 4 (.NET) 或 Rust 工具链或 Node 20+，外加 Gemini/Grok/Tripo 的 API key（画美术按张/按秒扣费，单次完整生成可能跑几小时），官方测试环境是 Ubuntu/Debian/macOS。所以量力而行的打开方式：

- ① **零成本读工艺**：仓库本身就是最好的教材——3 篇引擎指南是"AI 游戏开发的坑清单"，值回票价；② **零成本跑纯逻辑**：发布渲染、贴图集切片、codex 元数据生成本机就能真实跑（见下面实验）；③ **最低成本跑游戏**：选 **Babylon.js 路线**——只装 Node.js，游戏直接在浏览器里玩，看直播最省事；④ 完整 Godot 生成建议租一台带 GPU 的云 VM 挂着跑。另外这套"说明书写法"可以平移：给任何 Agent 项目写运行时清单时，"状态落盘 + 证据优先 + 静默陷阱清单"三板斧直接抄。

这个项目给了我什么

能力	说明
🎮 一句话生成完整游戏	发布游戏仓库 → Claude Code/Codex 进场 → 交付可运行项目 + 实机录像
🎨 AI 美术流水线	2D 图 (Gemini/Grok) 、3D 模型+骨骼动画 (Tripo) 、帧动画 (Grok 视频) 、抠图 (rembg) , 带成本意识
👁️ 视觉质检循环	无头引擎 + 截图/确定性录像, AI 看画面修问题, 编译过不算数
📖 三份"坑清单"指南	Godot/Bevy/Babylon 各一页, 专收静默失败陷阱, 任何引擎 AI 开发都能抄
🔧 发布渲染模式	一套 Markdown 模板 + 变量渲染出多引擎/多宿主口味, prompt 工程的工程化范本
💰 成本治理样板	花钱前确认、明码标价、免费档优先、资产尺寸登记表

动手实验：不动引擎，把"发布管线"真跑一遍

实验策略：本机没装 Godot 引擎、也没有画图 API key，但 Godogen 的本质是"发布说明书 + 纯逻辑工具"，这些完全可以真跑。共 3 个实验，全部在 `F:\reminder\godogen\exercise\` 完成，Python 3.14 + Pillow 12.3.0（已设 PYTHONUTF8=1）。

实验 1 • 用 publish_lite.py 复现发布管线 (godot × claude 口味)

操作 原版 `publish.sh` 依赖 `rsync` (Windows 没有)，我用 Python 标准库 `shutil` 写了等价的 `exercise/publish_lite.py` (复制 `asset-gen` → 调用上游 `render_dir.py` 做 `{VAR}` 替换 → 生成 `CLAUDE.md/godot.md/.gitignore`)，然后运行：

```
python publish_lite.py --engine godot --agent claude --out my-game
# 输出: published godot/claude -> my-game
```

目的 验证"thin runtime 发布模型"真实可用。 **实现了** 发布出完整游戏仓库骨架。 **验证成功** ✓ 目录结构完全正确：

```
my-game/
├─ CLAUDE.md           ← ${ENGINE_NAME}→Godot, /asset-gen 已替换 ✓
├─ godot.md            ← 引擎指南原样复制 ✓
├─ .claude/skills/asset-gen/ ← SKILL.md 里 ${RUNTIME_ASSET_DIR}→assets/ ✓
└─ .gitignore          ← godot 专属忽略规则 ✓
```

✓ 预期结果达成：渲染后的 `CLAUDE.md` 开头就是 "Build Godot game from a description"，技能路径全部指向 `.claude/skills/asset-gen/tools/`。

实验 2 • 换口味: babylon × codex + Codex 元数据生成

操作 `python publish_lite.py --engine babylon --agent codex --out my-game-codex`，再跑上游 `generate_codex_metadata.py`。 **目的** 验证"发布期渲染"的口味切换。 **实现了** 同一套模板的第二种口味。 **验证成功** ✓：

```
my-game-codex/
├─ AGENTS.md           ← 清单改名, $asset-gen 写法 ✓, Babylon.js ✓
├─ babylon.md + .agents/skills/asset-gen/
└─ .agents/skills/asset-gen/agents/openai.yaml ← 脚本从 frontmatter 生成 ✓
```

✓ `openai.yaml` 内容正确: `display_name "Asset Generator"`、`default_prompt "Use $asset-gen to..."`、`allow_implicit_invocation: true`——同一份 `SKILL.md`，两种宿主各取所需。

实验 3 • 跑真的 asset-gen 工具：贴图集切片 grid_slice.py

操作 先用 `make_atlas.py` (Pillow) 画一张 1024×1024 的 2×2 四色"AI 道具集"模拟生成产物，再调用上游真实工具切片：

```
python make_atlas.py
python ../repo/asset-gen/tools/grid_slice.py atlas.png -o sprites --grid 2x2 --names "sword,shield,
# 输出: {"ok": true, "cells": 4, "cell_size": "512x512", "paths": ["sprites\\sword.png", ...]}
python ../repo/asset-gen/tools/grid_slice.py atlas.png -o sprites_bad --grid 2x2 --names "a,b,c"
# 输出: {"ok": false, "error": "--names has 3 entries, grid is 4 cells"}
```

目的 验证 SKILL.md 里的"kit 省钱战术"真实可复现。 **实现了** 一张图切出 4 个独立 sprite (sword/shield/potion/helm, 各 512×512) 。 **验证成功** ☒ 成功/失败两条路径都输出了规范 JSON——这正是技能文档强调的"stdout 只出 JSON、进度走 stderr"的上下文卫生设计。

☒ **预期结果达成：4 张 sprite 尺寸全对 (512×512) ，参数个数不匹配时优雅报错不崩溃。**

没跑的部分 (如实记录)：Godot 引擎实机生成 (需装 Godot 4 .NET + 花钱 API key + 数小时)、Gemini/Grok/Tripo 付费生成、rembg 抠图 (需 onnxruntime-gpu) 。这些留待装好 Babylon.js 路线 (只需 Node) 或云 VM 时再战。

三句话带走

- 1. 源码即 prompt** —— Godogen 证明了纯 Markdown 的说明书可以是指挥 AI 干几小时活的产品，工程含量全在措辞里。
- 2. 证据优先于声称** —— AI 干活的验收标准必须是"看到它运行"，编译通过只值一分钱；无头引擎 + 截图录像就是 AI 的眼睛。
- 3. 只写模型推不出的** —— 给 AI 的指南别写常识，专收静默失败陷阱 + 钉死状态存哪 + 花钱闸门，剩下的信任模型。